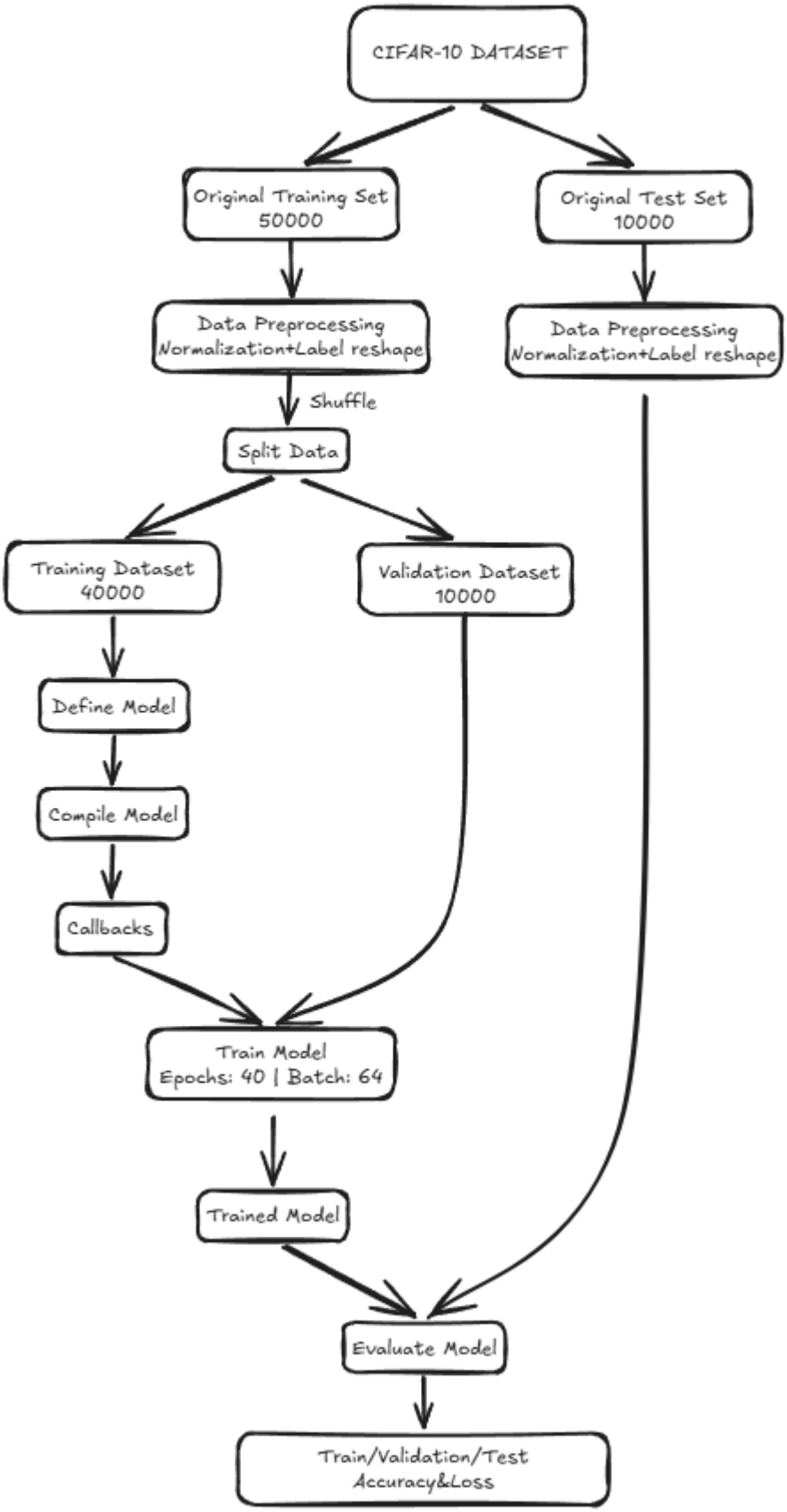
# 壹、架構

## 整體流程圖


CIFAR-10 DATASET
Original Training Set
50000
Original Test Set
10000
Data Preprocessing
Normalization+Label reshape
Data Preprocessing
Normalization+Label reshape
Shuffle
Split Data
Training Dataset
40000
Validation Dataset
10000
Define Model
Compile Model
Callbacks
Train Model
Epochs: 40 | Batch: 64
Trained Model
Evaluate Model
Train/Validation/Test
Accuracy&Loss

# 模型架構圖

```
Input (32,32,3)
        ↓
Data Augmentation
        ↓
Conv2D(64) + BN + ReLU      → (32×32×64)
        ↓
Conv2D(64) + BN + ReLU      → (32×32×64)
        ↓
MaxPooling2D                → (16×16×64)
        ↓
Dropout(0.25)
        ↓
Conv2D(128) + BN + ReLU     → (16×16×128)
        ↓
Conv2D(128) + BN + ReLU     → (16×16×128)
        ↓
MaxPooling2D                → (8×8×128)
        ↓
Dropout(0.30)
        ↓
Conv2D(256) + BN + ReLU     → (8×8×256)
        ↓
MaxPooling2D                → (4×4×256)
        ↓
Dropout(0.40)
        ↓
GlobalAveragePooling2D      → (256,)
        ↓
Dense(256, ReLU)
        ↓
Dropout(0.50)
        ↓
Dense(10, softmax)
```

# 貳、程式碼

## 前言：訓練與測試資料來源說明

使用 TensorFlow / Keras 內建的 cifar10.load_data() 載入 CIFAR-10 資料集作為影像分類實驗資料來源。CIFAR-10 為常用的影像分類基準資料集，包含 10 個類別的彩色圖片，每張影像大小為 32×32×3。原始資料集提供 50000 張訓練影像 與 10000 張測試影像，其中訓練資料用於模型學習，測試資料則保留作為最終評估模型泛化能力之用。

## 1. Prepare dataset

```
from tensorflow.keras.datasets import cifar10
(x_train_full, y_train_full), (x_test, y_test) = cifar10.load_data()
```

> 透過 cifar10.load_data() 載入 CIFAR-10，取得原始訓練集與測試集，作為後續影像分類的資料來源。

```
print("Original train shape:", x_train_full.shape)   # (50000, 32, 32, 3)
print("Original test shape:", x_test.shape)          # (10000, 32, 32, 3)
print("Original y_train shape:", y_train_full.shape) # (50000, 1)
print("Original y_test shape:", y_test.shape)        # (10000, 1)
```

> 透過印出 shape，可確認 CIFAR-10 的影像大小為 32×32×3，原始訓練集共有 50000 張，測試集共有 10000 張；標籤資料則以 (n,1) 的形式儲存。

```
x_train_full = x_train_full.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

> 將影像像素值由原本的 0~255 正規化到 0~1 範圍，可減少輸入值尺度差異，讓模型訓練過程更穩定，也有助於加快收斂。

```
y_train_full = y_train_full.reshape(-1)
y_test = y_test.reshape(-1)
```

> 原始載入之標籤資料形狀為 (n,1)，屬於二維陣列。為了配合 sparse_categorical_crossentropy 的輸入格式，這裡使用 reshape(-1) 將標籤轉換為一維向量 (n,)，讓每筆資料以單一整數類別編號表示。

```
indices = np.arange(len(x_train_full))
np.random.shuffle(indices)
x_train_full = x_train_full[indices]
y_train_full = y_train_full[indices]
```

> 先建立索引，再將索引順序隨機打亂，最後用同一組索引同步重排圖片與標籤。這樣做可在保留正確配對的前提下，打散資料原有順序，使後續切分出的訓練集與驗證集具有較均勻的分布。

```
(x_train, y_train), (x_val, y_val) = (
    (x_train_full[:40000], y_train_full[:40000]),
    (x_train_full[40000:], y_train_full[40000:]),
)
```

> 將已打亂的 50000 筆原始訓練資料切分為 40000 筆訓練集（training set）與 10000 筆驗證集（validation set）。

```
print("Train shape:", x_train.shape)         # (40000, 32, 32, 3)
print("Validation shape:", x_val.shape)      # (10000, 32, 32, 3)
print("Test shape:", x_test.shape)           # (10000, 32, 32, 3)
```

> 資料集前置準備完畢：訓練集有 40000 張大小為 32×32×3 的影像；驗證集有 10000 張大小為 32×32×3 的影像；測試集有 10000 張大小為 32×32×3 的影像。

## 2. Define CNN model

```
data_augmentation = Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomTranslation(0.1, 0.1),
    layers.RandomRotation(0.05),
], name="data_augmentation")
```

> 建立 Data Augmentation，透過隨機水平翻轉、隨機平移與隨機旋轉，增加訓練樣本的多樣性，使模型不易過度依賴固定位置、方向或姿態的特徵，進而降低過擬合風險。

```
model = Sequential([
    Input(shape=(32, 32, 3)),
    data_augmentation,

    Conv2D(64, (3, 3), padding='same'),
    BatchNormalization(),
    layers.ReLU(),
    Conv2D(64, (3, 3), padding='same'),
    BatchNormalization(),
    layers.ReLU(),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.25),

    Conv2D(128, (3, 3), padding='same'),
    BatchNormalization(),
    layers.ReLU(),
```

```
    Conv2D(128, (3, 3), padding='same'),
    BatchNormalization(),
    layers.ReLU(),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.30),

    Conv2D(256, (3, 3), padding='same'),
    BatchNormalization(),
    layers.ReLU(),
    MaxPooling2D(pool_size=(2, 2)),
    Dropout(0.40),

    GlobalAveragePooling2D(),
    Dense(256, activation='relu'),
    Dropout(0.50),
    Dense(10, activation='softmax')
])
```

| 區塊 | 內容 | 主要目的 |
|---|---|---|
| Input + Augmentation | Input、水平翻轉、平移、旋轉 | 提升資料多樣性 |
| Convolution Block 1 | 2 層 Conv64 + BN + ReLU + Pool + Dropout | 提取初階特徵 |
| Convolution Block 2 | 2 層 Conv128 + BN + ReLU + Pool + Dropout | 提取中階特徵 |
| Convolution Block 3 | 1 層 Conv256 + BN + ReLU + Pool + Dropout | 提取高階特徵 |
| Classifier | GAP + Dense256 + Dropout + Dense10 | 完成分類輸出 |

> 此模型先透過資料增強模組提升訓練樣本多樣性，再利用三個卷積區塊逐步提取影像特徵。第一、二個卷積區塊各使用兩層卷積，可更充分學習局部紋理與中階特徵；第三個卷積區塊則將通道數提升至 256，以提取更高階的語意資訊。卷積區塊中結合 Batch Normalization、ReLU、Max Pooling 與 Dropout，可提升訓練穩定性並降低過擬合風險。最後透過 Global Average Pooling 減少參數數量，再以全連接層整合特徵，並使用 softmax 輸出 CIFAR-10 十個類別的分類機率。

## 3. Compile model

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

> 決定訓練方式與評估指標：設定模型要用 Adam 來學習、用 sparse categorical crossentropy 來計算分類錯誤、並用 accuracy 來顯示分類表現。

## 4. Callbacks

```
os.makedirs("outputs", exist_ok=True)
```

> 建立輸出資料夾 outputs，用來統一存放模型、結果文字檔與訓練曲線等輸出內容。

```
early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=8,
    restore_best_weights=True,
    verbose=1
)
```

> EarlyStopping 會監控驗證集損失 val_loss。當驗證表現連續數個 epoch 沒有改善時，自動停止訓練，避免浪費時間；restore_best_weights=True 則表示訓練結束後恢復到最佳權重。

```
reduce_lr = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=3,
    min_lr=1e-5,
    verbose=1
)
```

> ReduceLROnPlateau 用來在驗證集表現停滯時，自動降低 learning rate。這樣可使模型在訓練後期以較小步伐繼續調整參數，幫助模型更穩定地收斂。

```
checkpoint = ModelCheckpoint(
    "outputs/best_model.keras",
    monitor="val_loss",
    save_best_only=True,
    verbose=1
)
```

> ModelCheckpoint 會根據 val_loss 自動儲存驗證表現最佳的模型，確保最終保留下來的是訓練過程中表現最好的版本，而不只是最後一輪的模型。

```
csv_logger = CSVLogger('outputs/training_history.csv', append=False)
```

> CSVLogger 用來記錄每個 epoch 的訓練歷程，包含 loss、accuracy、val_loss 與 val_accuracy，方便後續分析與繪製訓練曲線。

```
with open("outputs/model_summary.txt", "w", encoding="utf-8") as f:
    model.summary(print_fn=lambda x: f.write(x + "\n"))
```

> 將 model.summary() 輸出為文字檔 model_summary.txt，用來保存模型各層結構、輸出尺寸與參數數量。

## 5. Fit model

```
history = model.fit(
    x_train, y_train,
    validation_data=(x_val, y_val),
    epochs=40,
    batch_size=64,
    callbacks=[early_stopping, reduce_lr, csv_logger, checkpoint],
    verbose=1
)
```

> 使用 model.fit() 將訓練資料 x_train、y_train 輸入模型進行學習，並同時指定 validation_data=(x_val, y_val) 作為每個 epoch 結束後的驗證依據。訓練最多進行 40 個 epochs，每次以 64 筆樣本作為一個 mini-batch 進行參數更新。訓練過程中結合 EarlyStopping、ReduceLROnPlateau、CSVLogger 與 ModelCheckpoint 等 callbacks，以達到自動停止訓練、調整學習率、記錄訓練歷程與儲存最佳模型之目的。最終 model.fit() 所回傳的 history 物件會保存每個 epoch 的 loss 與 accuracy。

## 6. Evaluate model

```
train_loss, train_acc = model.evaluate(x_train, y_train, verbose=0)
val_loss, val_acc = model.evaluate(x_val, y_val, verbose=0)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0)

print("Train Accuracy:", train_acc)
print("Validation Accuracy:", val_acc)
print("Test Accuracy:", test_acc)
```

> 使用 model.evaluate() 分別對訓練集、驗證集與測試集進行評估，取得各資料集上的 loss 與 accuracy。

```
with open("outputs/final_result.txt", "w", encoding="utf-8") as f:
    f.write(f"Train Accuracy: {train_acc:.6f}\n")
    f.write(f"Validation Accuracy: {val_acc:.6f}\n")
    f.write(f"Train Loss: {train_loss:.6f}\n")
    f.write(f"Validation Loss: {val_loss:.6f}\n")
    f.write(f"Test Loss: {test_loss:.6f}\n")
    f.write(f"Test Accuracy: {test_acc:.6f}\n")
```

> 將最終評估結果寫入 outputs/final_result.txt，保存 train / validation / test 的 accuracy 與 loss。

```
model.save("outputs/cifar10_cnn_improved.keras")
```

> 保存最終模型。

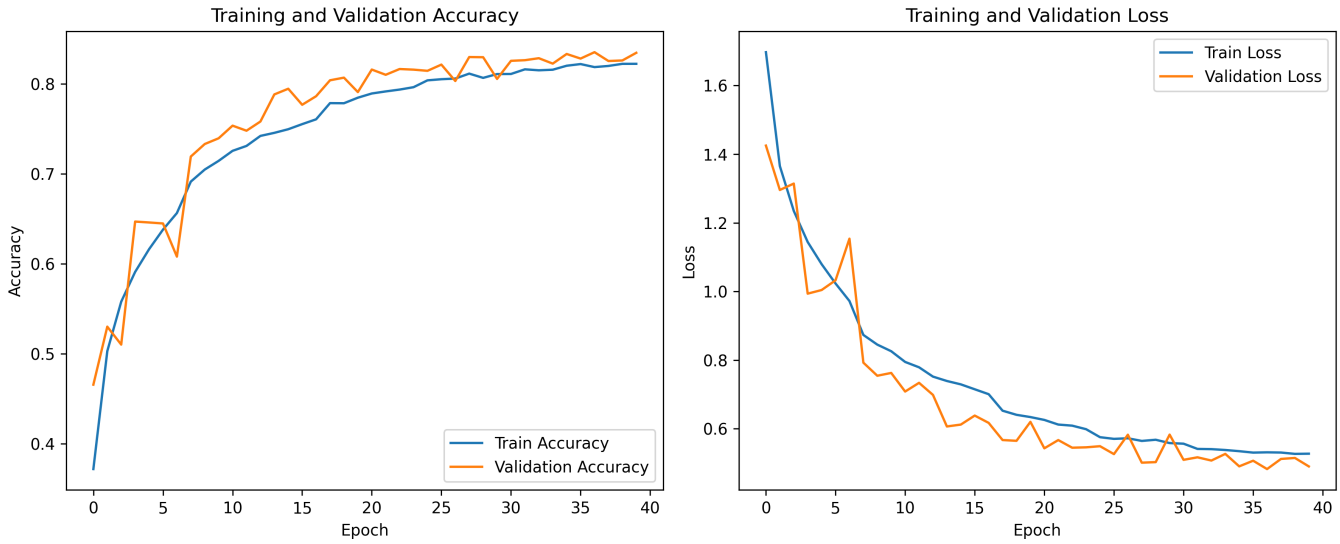
### Plot Accuracy and Loss

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Training and Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.savefig("outputs/accuracy_loss_curve.png", dpi=300)
plt.show()
```

> 繪製訓練曲線。

# 參、結果

## final_result.txt

```
Train Accuracy: 0.864175
Validation Accuracy: 0.835000
Train Loss: 0.398744
Validation Loss: 0.482634
Test Loss: 0.496428
Test Accuracy: 0.832900
```

## accuracy_loss_curve.png